SD242 · VERIFICAÇÃO DE SISTEMAS DIGITAIS · TRABALHO FINAL

Ambiente UVM para o top-level RISC-V + DMA + Systolic Array

Cada grupo deve construir um ambiente de verificação UVM, com checagem automática e cobertura funcional, para o top-level do seu Projeto Final, e apresentá-lo pronto e rodando no Xcelium ao final da disciplina.

DUT	METODOLOGIA	SIMULADOR	APRESENTAÇÃO
Top-level do grupo	**UVM 1.2 em SystemVerilog**	**Cadence Xcelium + SimVision**	`data a definir`

1 Objetivo

O DUT é o sistema especificado no documento *RISC-V com Coprocessador de Multiplicação Matricial e Controlador DMA*: processador RV32I, controlador de DMA, coprocessador de multiplicação matricial em systolic array e memória compartilhada, integrados em um único top-level. O grupo verifica o seu próprio projeto.

Este trabalho detalha, do ponto de vista da disciplina de Verificação, o item 7 e o "UVM Integrado" da Entrega 4 daquele documento. O que se avalia aqui é o ambiente: como as transações foram modeladas, como os componentes foram organizados e conectados, e se a checagem é de fato automática.

Ao final, o grupo deve ser capaz de:

- modelar o estímulo e a resposta do sistema como transações e sequences;
- implementar um agente com sequencer, driver e monitor para um DUT cujo estímulo é o conteúdo das memórias, aplicado por meio do firmware, e não um protocolo de pinos;
- conectar monitor, scoreboard e demais componentes por TLM;
- decidir automaticamente, sem inspeção de waveform, se a matriz C produzida pelo hardware está correta;
- medir, com cobertura funcional, o quanto os testes exercitaram o sistema;
- executar testes dirigidos e aleatórios por linha de comando, com seed reprodutível.

2 A natureza do estímulo neste DUT

O top-level deste trabalho quase não tem pinos funcionais: em geral, clock, reset e uma linha de interrupção. A CPU está dentro do DUT, e é ela que programa os registradores do DMA e do acelerador. O testbench não pode dirigir esse barramento interno sem passar por cima do processador.

O estímulo, portanto, é o **conteúdo das memórias**: as matrizes A e B e os parâmetros da operação. O firmware não é o estímulo, e sim o meio pelo qual a CPU o aplica ao restante do sistema. A resposta é a matriz C que o DMA escreve na memória de dados.

> DECISÃO DO GRUPO
>
> Cabe ao grupo decidir como representar esse estímulo e essa resposta em transações, e em que nível de abstração. A escolha deve ser justificada na apresentação.

3 Requisitos obrigatórios

O ambiente deve conter todos os itens abaixo. Nomes de classes e arquivos são livres, desde que consistentes.

ITEM	MÍNIMO EXIGIDO
Transação	Uma ou mais classes derivadas de `uvm_sequence_item` que modelem o estímulo e a resposta do sistema. Campos `rand`, constraints e nível de abstração são decisão do grupo. Deve implementar `convert2string` e comparação (`do_compare` ou field macros).
Sequences	Biblioteca com sequences dirigidas e aleatórias, usando `start_item` / `randomize` / `finish_item`.
Sequencer	`uvm_sequencer` parametrizado com a transação, criado pelo agente e conectado ao driver.
Driver	`uvm_driver` que obtém itens com `get_next_item` / `item_done` e os aplica ao DUT por meio de um virtual interface obtido no `uvm_config_db` (ver seção 6).
Monitor	Componente passivo que observa o DUT, detecta o término da operação, recolhe a matriz C e publica uma transação em um `uvm_analysis_port`. O monitor não consulta o driver nem a sequence.
Agente	`uvm_agent` que encapsula sequencer, driver e monitor, com objeto de configuração (virtual interface, `is_active`, parâmetros como `ARRAY_SIZE` e timeout).
TLM	Toda comunicação entre componentes é feita por portas TLM conectadas no `connect_phase`: `seq_item_port` entre driver e sequencer, e analysis port do monitor para o scoreboard e para a cobertura (direto, por `uvm_tlm_analysis_fifo` ou via reference model). Nenhum componente acessa outro por handle direto para trocar dados.
Scoreboard	Compara, elemento a elemento, a matriz C observada com a esperada e reporta divergências com `` `uvm_error `` indicando linha, coluna, valor esperado e valor obtido. Ao final, informa quantas operações foram comparadas e acusa operações pendentes (ver seção 7).

ITEM	MÍNIMO EXIGIDO
Cobertura funcional	Modelo de cobertura definido pelo grupo, com covergroups amostrados a partir das transações publicadas pelo monitor. O resultado deve ser coletado na regressão e apresentado (ver seção 7).
Env e testes	`uvm_env` que cria e conecta os componentes; classe base de teste e testes derivados selecionados por `+UVM_TESTNAME`. O plano de testes é responsabilidade do grupo: deve ter testes dirigidos e aleatórios e atender ao item 7 do projeto (diferentes dimensões de matrizes e diferentes valores de dados). Criação de componentes e objetos sempre pela factory (`type_id::create`).
Execução	A forma de compilar e executar é livre (script, Makefile ou outra). Qualquer teste deve poder ser compilado e rodado pela linha de comando, com seed e coleta de cobertura selecionáveis, terminando com veredito explícito no log (`TEST PASS` / `TEST FAIL`, contagem de `UVM_ERROR` e `UVM_FATAL`).

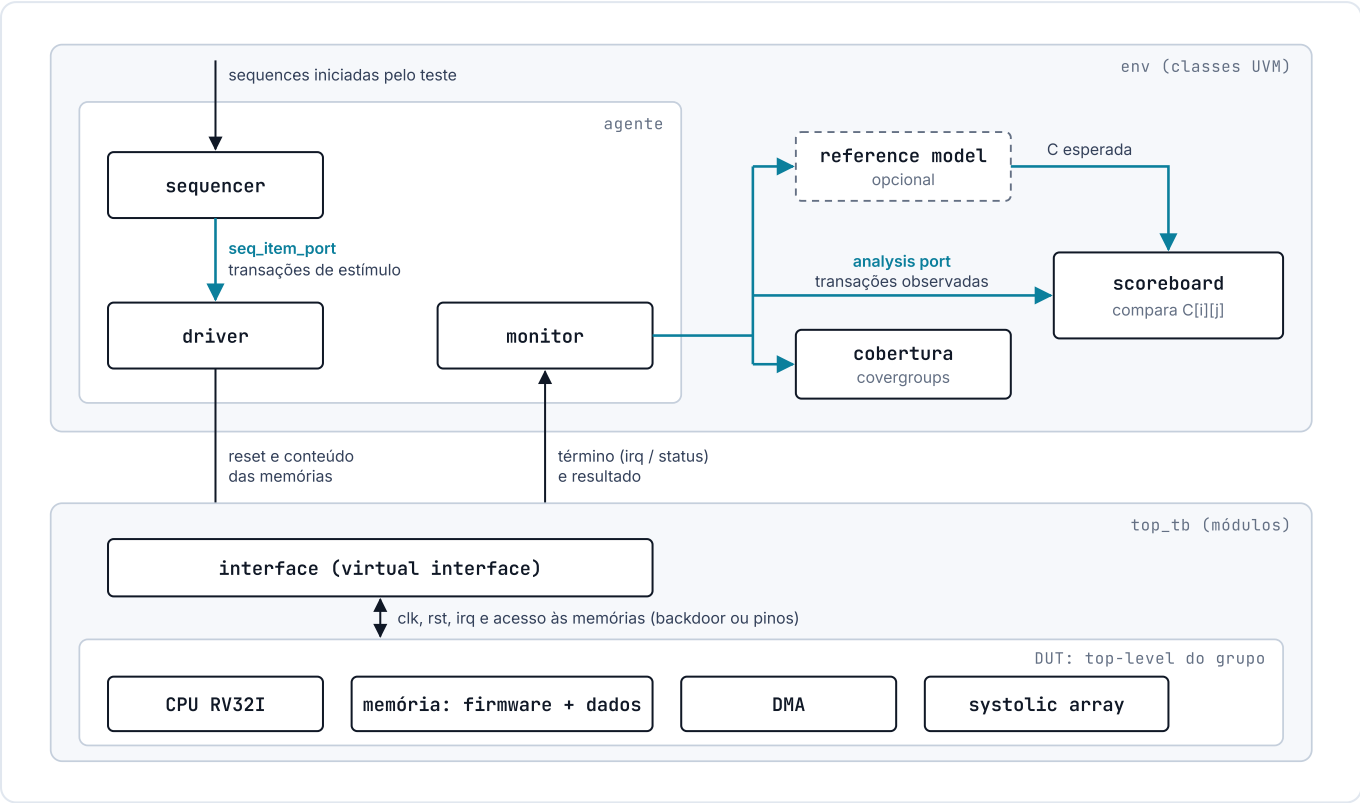
4 Itens desejáveis OPCIONAIS

Não são exigidos para a nota integral, mas contam como bônus (seção 11) e aproximam o ambiente de um testbench profissional.

ITEM	O QUE SE ESPERA
Reference model	Componente UVM separado do scoreboard: recebe por TLM a transação observada pelo monitor, calcula a matriz C esperada e a publica para o scoreboard. Pode ser escrito em SystemVerilog ou reaproveitar, via DPI-C, o golden model em C/C++ da Entrega 1 do projeto.
Monitor de barramento	Agente passivo no barramento interno (CPU ↔ registradores, ou DMA ↔ memória) que verifique a autonomia do DMA: depois do comando de início, nenhuma escrita na região de C parte da CPU.
Mais de um `ARRAY_SIZE`	Regressão passando em pelo menos duas configurações da malha, selecionáveis na linha de comando.
Latência	Monitor mede o número de ciclos entre o início e o término de cada operação e o registra na transação. O mesmo número serve à análise de desempenho do relatório do projeto.

5 Arquitetura de referência

A figura mostra uma organização que atende aos requisitos. Ela é uma sugestão: o grupo pode adotar outra, desde que justifique as escolhas na apresentação.



Em ciano, as conexões TLM exigidas. A caixa tracejada é opcional: sem o reference model, o scoreboard obtém o valor esperado por conta própria (seção 7). Driver e monitor só alcançam o DUT pelo virtual interface. Na alternativa B da seção 6, a memória fica fora do DUT e é modelada pelo driver.

6 Como o estímulo chega ao DUT

Em qualquer arquitetura, a configuração do DMA e do acelerador é feita pelo firmware executado na CPU. O que o ambiente controla é o conteúdo das memórias. Há duas formas de fazer isso; o grupo escolhe uma e justifica a escolha.

ALTERNATIVA	COMO FUNCIONA	CONSEQUÊNCIA
A · Carga das memórias por backdoor	As memórias ficam dentro do top-level. Com o sistema em reset, o driver escreve nelas o conteúdo de cada transação, por tarefas da interface. Depois libera o reset, deixa o firmware executar e aguarda o término (interrupção ou registrador de status).	Driver simples, que só dirige o reset. O monitor detecta o término pelos pinos e lê o resultado da memória também por backdoor.

ALTERNATIVA	COMO FUNCIONA	CONSEQUÊNCIA
B · Driver como memória	As memórias ficam fora do top-level verificado, que expõe os pinos de endereço, dado e controle. O driver faz o papel da memória: responde às leituras da CPU e do DMA com instruções e dados, e armazena as escritas.	O driver dirige pinos de verdade, mas passa a ser reativo: quem inicia cada acesso é o DUT. O monitor reconstrói as transações a partir do barramento, sem backdoor. Dá mais trabalho e exige que o top-level tenha essa interface.

Nas duas alternativas, o firmware precisa conhecer os parâmetros de cada operação. Também aqui há duas abordagens:

ABORDAGEM	COMO FUNCIONA	CONSEQUÊNCIA
Firmware genérico	Um único binário lê dimensões e endereços de um bloco de parâmetros em posição fixa da memória de dados, fornecido pelo ambiente a cada operação.	Qualquer operação aleatória roda sem recompilar o firmware. Várias operações por simulação.
Firmware por teste	Cada teste carrega um `.hex` diferente (por plusarg), com dimensões e endereços fixos no programa.	Mais simples de começar, mas limita a randomização ao que está previsto nos binários.

REGRAS PARA O ACESSO AO DUT

- A configuração dos registradores do DMA e do acelerador deve ser feita pelo firmware executado na CPU. No teste de top-level não vale `force` nem escrita backdoor nesses registradores.
- Acesso backdoor é permitido apenas para carregar e ler as memórias de instruções e de dados.
- Referências hierárquicas ao DUT ficam restritas ao `top_tb` e à interface. As classes UVM só enxergam o virtual interface.
- O monitor obtém os dados da operação a partir do DUT (memória ou barramento), e não do item do driver. Assim um erro do driver ou do firmware também é detectado.
- Toda espera tem timeout. Uma operação que não termina deve gerar `` `uvm_error `` ou `` `uvm_fatal ``, nunca uma simulação travada.

7 Checagem automática e cobertura

7.1 Scoreboard

O veredito de cada teste vem do scoreboard, não da leitura de waveform nem de comparação visual do log. O valor esperado de C pode vir de uma destas fontes:

FONTE DO VALOR ESPERADO	DESCRIÇÃO	SITUAÇÃO
Cálculo no scoreboard	Uma função do próprio scoreboard multiplica as matrizes A e B recebidas na transação observada.	Aceita
Arquivo do golden model	O modelo de referência da Entrega 1 gera arquivos com A, B e C; a sequence lê A e B e o scoreboard lê C.	Aceita
Reference model por TLM	Componente separado que prevê C e a publica para o scoreboard.	Desejável

Em qualquer caso, o valor esperado não pode ser derivado da saída do próprio DUT. O scoreboard deve ainda:

- respeitar a largura do acumulador definida no projeto (truncamento ou saturação, com ou sem sinal);
- verificar, no `check_phase` ou no `report_phase`, que não restaram operações sem comparação;
- imprimir um resumo final com o número de operações comparadas, de acertos e de divergências.

7.2 Cobertura funcional

O grupo define o próprio modelo de cobertura, a partir do que considera necessário exercitar no sistema, e deve saber explicar cada coverpoint. Exige-se que:

- os covergroups sejam amostrados a partir das transações publicadas pelo monitor, em um componente ligado ao analysis port (`uvm_subscriber` ou equivalente), e nunca a partir do item do driver;
- o modelo tenha pelo menos um `cross` entre coverpoints;
- a coleta possa ser habilitada pela linha de comando e o resultado fique visível no log ou no IMC;
- o grupo apresente o modelo de cobertura, o percentual atingido pela regressão e a explicação do que ficou descoberto.

8 Organização e execução

O ambiente fica no mesmo repositório GitHub do Projeto Final, separado do RTL. Recomenda-se uma estrutura em camadas, com um pacote por camada (agente → ambiente → testes):

ESTRUTURA SUGERIDA

```
verif/
├── agents/<agente>/     interface, transação, config, driver, monitor, sequencer,
│                        agente, biblioteca de sequences, pacote
├── env/                 scoreboard, cobertura, env, pacote (+ reference model)
├── tests/               classe base de teste, testes, pacote
└── top/                 top_tb.sv: clock, reset, DUT, interface, run_test()
firmware/                fonte e binário (.hex) carregado na memória de instruções
sim/                     scripts de compilação, simulação e regressão
```

A automação da compilação, da simulação e da regressão fica a critério do grupo. O `README` da pasta de verificação descreve o DUT em um parágrafo, a função de cada arquivo, os comandos para compilar, rodar um teste, abrir o waveform e rodar a regressão, e a lista de testes com o resultado esperado de cada um. Uma pessoa de fora do grupo deve conseguir rodar a regressão seguindo apenas o README.

9 Entregáveis

ENTREGÁVEL	CONTEÚDO	PRAZO
Repositório	Ambiente UVM, firmware, scripts e README, no repositório do projeto. O commit da entrega deve ser identificado por uma tag.	`a definir`
Log de regressão	Saída da regressão com todos os testes do grupo, a seed de cada execução, o veredito e a cobertura funcional obtida.	`a definir`
Slides	Arquivo PDF com os slides da apresentação (seção 10).	`a definir`
Apresentação e demonstração	Defesa oral com demonstração ao vivo no Xcelium (seção 10).	`a definir`

10 Apresentação e demonstração ao vivo

Cada grupo tem `20 min`: cerca de 9 para os slides, 8 para a demonstração e 3 para perguntas. Qualquer integrante pode ser chamado a explicar qualquer parte do ambiente.

O conteúdo e a organização dos slides são livres.

Roteiro da demonstração no Xcelium

A demonstração é feita ao vivo, no servidor da disciplina, a partir do repositório entregue.

1. **Compilação limpa.** Remover os artefatos e compilar do zero.
2. **Teste dirigido em batch.** Rodar um teste dirigido e mostrar no log a topologia impressa pelo UVM, as mensagens do scoreboard e o veredito.

3. **Teste aleatório com seed escolhida na hora.** Rodar um teste aleatório com uma seed indicada pelo professor.
4. **Cobertura funcional.** Mostrar o resultado da cobertura, no log ou no IMC, e comentar o que ficou descoberto.
5. **Uma operação completa no SimVision.** Mostrar no waveform a configuração dos registradores pela CPU, as leituras do DMA, o processamento no systolic array, a escrita de C e a sinalização de término.
6. **O ambiente detectando um erro.** Injetar uma falha no RTL ou no firmware (um `` `define `` ou plusarg preparado para isso) e mostrar o scoreboard apontando o elemento divergente e o teste terminando em `TEST FAIL`.

> ANTES DO DIA
>
> Ensaiem a demonstração no mesmo servidor e com o mesmo usuário que será usado na apresentação. Tenham uma gravação de tela da demonstração como contingência: ela só substitui a execução ao vivo se a infraestrutura (servidor ou licença) falhar.

11 Critérios de avaliação

CRITÉRIO	O QUE É OBSERVADO	PESO
Ambiente UVM	Modelagem das transações e sequences; driver, sequencer e monitor; conexões TLM; organização em camadas, uso de factory e `config_db`.	35%
Checagem e cobertura	Scoreboard com checagem automática e completa; modelo de cobertura funcional e resultado atingido; testes dirigidos e aleatórios coerentes com o que o grupo se propôs a verificar.	25%
Apresentação	Clareza da exposição e domínio de todos os integrantes sobre o próprio ambiente.	10%
Demonstração ao vivo	Os seis passos do roteiro da seção 10 executados ao vivo, sem editar código durante a demonstração.	30%
Total		**100%**

Os itens desejáveis da seção 4 valem até 10% de bônus. Um ambiente que não compila ou não roda no dia não pontua no critério “Demonstração ao vivo”.

12 Estratégia recomendada

- **Fechem o caminho completo cedo.** Primeiro um único teste simples atravessando sequence → driver → DUT → monitor → scoreboard. Só depois randomização, cobertura, mais testes e itens opcionais.
- **Comecem antes de a CPU estar integrada.** No subsistema DMA + memória + acelerador (Entrega 3 do projeto), um driver pode programar os registradores diretamente pelo barramento. No top-level, um

segundo driver, selecionado por factory override, deixa essa tarefa para o firmware. Transação, sequences, monitor e scoreboard permanecem os mesmos.

- **Deixem a cobertura guiar os testes.** O que o relatório mostra como descoberto indica que estímulo ainda falta.
- **Testem o testbench.** Antes de confiar em um `TEST PASS`, injetem um erro e confirmem que o scoreboard o acusa. O mesmo mecanismo serve ao último passo da demonstração.
